

Resume-to-Job-Description Fit Scorer — One-Page Design

Explanation

1. Design parameter chosen: calibration strength

I chose a **15% calibration strength**. The function is $50 + 0.85 \times (\text{raw score} - 50)$, pulling scores 15% toward the midpoint of 50. This reduces exaggerated score differences while preserving ordering. In the three-sample offline demonstration, Resume A and Resume B had raw scores of 78 and 74 (4-point gap); after calibration they became 73.8 and 70.4 (3.4-point gap). The 15-point similarity threshold is a project-level guardrail, not an industry standard. The demonstration uses controlled raw scores rather than claiming statistical validation.

2. One actual observed failure and its cause

An unreadable blank PDF produced a controlled validation error because the parser extracted no text; the request stops rather than sending an empty resume to the LLM. OCR is not implemented, so scanned image-only resumes remain a limitation. During live Gemini testing, the first LLM request succeeded but the second returned **HTTP 429 RESOURCE_EXHAUSTED** because the external Gemini free-tier request quota was exhausted. The application converts LLM failures into a controlled 502 response and does not blindly retry quota errors. This was an external API quota limitation, not an application crash.

3. Metric tracked and what it told me

The API records **end-to-end latency in milliseconds** so processing cost can be observed across parsing, LLM calls, and scoring. I also track the calibration gap between similar sample resumes as a stability diagnostic. Resume A/B moved from a 4.0-point raw gap to a 3.4-point calibrated gap. These are engineering diagnostics, not substitutes for a human-labeled accuracy benchmark.

4. What is not finished and next steps

The core local workflow is implemented, but production-level validation is not finished. Remaining work includes OCR for scanned PDFs, evaluation on a larger human-labeled resume/JD dataset, stronger agreement and ranking metrics, regression tests, more extensive structured-output validation, and deployment. The current two-call architecture already batches all five criterion scores into one scoring call; further work should focus on reliability and empirical evaluation.

5. Honest status

The FastAPI pipeline, PDF/TXT parsing, structured criterion extraction, per-criterion scoring, configurable weights, calibration demonstration, and edge-case handling are implemented and locally tested. A full live two-call Gemini analysis was only partially demonstrated because the second call was blocked by the external quota. The main remaining uncertainty is scoring accuracy on a sufficiently large, human-labeled dataset.

Technology: Python, FastAPI, Pydantic, Gemini, PyMuPDF. **Weights:** skills 40%, experience 25%, education 10%, responsibilities 15%, domain 10%. Weights are configuration and normalized before use.